

11. Cómo ejecutar el proyecto

Este documento explica cómo poner en funcionamiento VentaGamer. La forma recomendada es con **Docker**, que levanta el sistema completo (frontend, backend, base de datos y motor de IA) con un solo comando y sin instalar ninguna otra dependencia. Al final se describe también el modo de desarrollo local.

11.1. Requisitos

Requisito	Detalle
Docker con Compose v2	Único requisito para el modo recomendado. En Linux: <code>docker.io</code> + <code>docker-compose-v2</code> ; en Windows/macOS: Docker Desktop.
Espacio en disco	~5 GB libres (imágenes de los contenedores + modelo de IA de ~2 GB).
Puertos libres	8080 (web), 5050 (API), 1434 (SQL Server), 11434 (Ollama).

11.2. Modo recomendado: todo con Docker

El repositorio incluye dos scripts en la raíz que automatizan la puesta en marcha:

Paso 1 — Preparación (una sola vez)

```
cd VentaGamer-Modern
sudo ./setup-docker.sh
```

Este script habilita el servicio de Docker, agrega el usuario actual al grupo `docker` (para no necesitar `sudo` en adelante) y prepara el archivo de configuración `.env` con los valores de desarrollo, incluido el modelo de IA por defecto (`qwen2.5:3b`). Tras ejecutarlo, aplicar el nuevo grupo con `newgrp docker` (o cerrando y reabriendo la sesión).

Paso 2 — Levantar el sistema

```
./levantar.sh
```

El script ejecuta `docker compose up -d --build --wait`, que:

1. Construye las imágenes del backend (compilación .NET) y del frontend (build de Vite + nginx).
2. Levanta SQL Server y Ollama, esperando a que sus verificaciones de salud den positivo.
3. Levanta la API, que **aplica las migraciones y puebla los datos iniciales automáticamente** (roles, permisos, usuarios demo, 20 productos, traducciones).

4. Descarga el modelo de IA la primera vez (~2 GB; puede tardar varios minutos).

La primera ejecución completa puede llevar varios minutos (descarga de imágenes, compilación y modelo). Las siguientes son casi inmediatas.

Resultado

Servicio	URL
Aplicación web	http://localhost:8080
API (Swagger, solo en modo desarrollo)	http://localhost:5050/swagger
Verificación de salud de la API	http://localhost:5050/api/health
SQL Server	localhost, 1434 (usuario sa)
Ollama	http://localhost:11434

Usuarios de demostración

El sistema se inicializa con cuentas de prueba que cubren todos los perfiles:

Usuario	Contraseña	Rol	Qué permite probar
cliente	Cliente123!	Cliente (User)	Catálogo, carrito, checkout, comprobante PDF, chat IA
juan	Juan123!	Cliente (User)	Segundo cliente (para ver pedidos de varios usuarios)
admin	Admin123!	Administrador	ABM de productos, usuarios, roles y permisos, configuración IA, todas las compras
webmaster	WebMaster123!	WebMaster	Bitácora de auditoría, backups, verificación de integridad
tester	Tester123!	Tester	Perfil mixto de consulta (QA)

Comandos útiles

```

docker compose ps          # Estado de los 4 contenedores
docker compose logs -f     # Logs en vivo (Ctrl+C para salir)
docker compose logs backend # Logs solo de la API
docker compose down        # Detener todo (los datos persisten en los volúmenes)
docker compose down -v     # Detener y BORRAR los datos (reinicio de cero)
    
```

11.3. Recorrido de prueba sugerido

Para verificar el sistema completo en unos minutos:

1. **Como visitante:** abrir <http://localhost:8080>, navegar el catálogo, buscar y filtrar por categoría (sin iniciar sesión).
2. **Como cliente** (`cliente` / `Cliente123!`): agregar productos al carrito, modificar cantidades, confirmar la compra y **descargar el comprobante PDF** desde el detalle del pedido. Probar el asistente **GG** (botón flotante): preguntar por ejemplo "¿qué consolas tenés en stock?".
3. **Como administrador** (`admin` / `Admin123!`): crear o editar un producto en `/admin/products` ; revisar todas las compras en `/admin/orders` ; ver roles y permisos en `/admin` ; verificar el estado del asistente en `/admin/ai` .
4. **Como webmaster** (`webmaster` / `WebMaster123!`): revisar en `/audit` la bitácora (deberían aparecer las operaciones recién realizadas) y generar un backup en `/maintenance` .
5. **Responsive:** achicar la ventana o abrir desde un móvil — el menú pasa a hamburguesa y las grillas se reorganizan.
6. **Multi-idioma:** cambiar el idioma con el selector del encabezado (es/en/pt).

11.4. Modo alternativo: desarrollo local

Para trabajar sobre el código con recarga en caliente se puede ejecutar cada pieza por separado. Requiere: **.NET 8 SDK**, **Node.js 20+** y Docker (solo para la base y Ollama).

```
# 1. Base de datos y Ollama en Docker
docker compose up -d sqlserver ollama

# 2. Backend (puerto 5050)
cd backend
dotnet restore
dotnet run --project src/VentaGamer.Api

# 3. Frontend (puerto 5173, en otra terminal)
cd frontend
npm install
npm run dev
```

En este modo la aplicación queda en <http://localhost:5173>; el servidor de Vite reenvía `/api` y `/hubs` al backend en el puerto 5050. Swagger queda disponible en <http://localhost:5050/swagger>.

11.5. Solución de problemas frecuentes

Problema	Causa probable	Solución
<code>permission denied</code> <code>... docker.sock</code>	El usuario no pertenece al grupo <code>docker</code> .	Ejecutar <code>sudo ./setup-docker.sh</code> y luego <code>newgrp docker</code> (o reiniciar la sesión).
Un puerto ya está en uso	Otro servicio ocupa 8080/5050/1434/11434.	Liberar el puerto o cambiar el mapeo en <code>docker-compose.yml</code> .
El backend se reinicia al arrancar	SQL Server todavía estaba iniciándose.	Esperar: las dependencias con healthcheck lo reintentan solos. Ver <code>docker compose logs backend</code> .
El asistente GG figura "offline"	El modelo aún se está descargando, u Ollama no está corriendo.	Verificar con <code>docker compose logs ollama</code> ; el script <code>levantar.sh</code> descarga el modelo automáticamente.
La web carga pero las llamadas fallan	El backend no está sano.	<code>docker compose ps</code> debe mostrar los 4 contenedores "healthy"; revisar logs de la API.
Se quiere reiniciar los datos de cero	—	<code>docker compose down -v &&</code> <code>./levantar.sh</code> (borra la base y vuelve a sembrar los datos demo).